

CASE STUDY

Design and Implementation of a Context-Aware Zero Trust Authorization Middleware for REST APIs

*Demonstrated via **TransitOps** - a Smart Fleet Operations Platform*

The Problem Statement:

Traditional web applications authenticate a user once at login and then implicitly trust every request that follows on that session or token. This “trust once, allow always” model does not detect a stolen session used from a new device, an unfamiliar location, or an unusual hour — the exact blind spot that Zero Trust Architecture (NIST SP 800-207) was created to close. Commercial platforms that solve this (Zscaler, Okta, Microsoft Entra) are enterprise-priced switchboards that are impractical for a student, a small team, or an internship to build from scratch, which makes the underlying decision logic hard to learn hands-on.

Real-life Scenario Observed:

The problem becomes particularly significant in fleet operations, where actions such as dispatching a route, changing a driver status, modifying a vehicle, recording financial activity, or changing a security policy can affect safety, cost, compliance, and service continuity. Login-only security is insufficient because the context of a request can change after authentication.

The problem, therefore, is to **design and demonstrate a lightweight, reusable authorization layer that re-evaluates trust on every single request** — not just at login — using continuous, context-aware risk scoring and policy evaluation, without requiring production-grade infrastructure to prove the concept.

What Must the solution do??

The system should evaluate access continuously at request level, use an explainable 0–100 risk score, produce clear outcomes such as **ALLOW, STEP-UP, READ-ONLY, or BLOCK**, record successful and denied activity in an auditable stream, and provide administrators with a way to safely test security scenarios through simulation.

Finding a Solution

To address the limitations of login-only authorization, we built a **Zero Trust Continuous Authorization (ZTCA) Middleware** — a self-contained Express module that intercepts every API request, independent of the business logic it protects.

The middleware constructs a fresh request context using identity, role, device fingerprint, browser and operating system, location, timestamp, endpoint, HTTP method, and action information. This context is evaluated by a risk engine that produces an explainable 0–100 risk score. The resulting risk is then evaluated against externally defined policies before a final authorization decision is enforced.

The decision layer produces one of four outcomes — **ALLOW, STEP-UP, READ-ONLY, or BLOCK** — and attaches a human-readable reason to the result. Every decision is persisted in an audit store so that the action, context, risk factors, policy evaluation, and outcome can be inspected later through the Security Admin Panel.

To generate realistic traffic for this security layer, **TransitOps** was developed as a full-stack Smart Fleet Operations Platform covering vehicles, drivers, trips, fuel, maintenance, expenses, reports, and operational activity. TransitOps therefore serves as the demonstration environment rather than the primary security contribution.

The overall approach follows the request lifecycle:

Browser Action → Express API → ZTCA Context Collection → Risk Evaluation → Policy Decision → Audit Persistence → Enforcement → Admin Panel Reflection

Because the middleware evaluates the request rather than the internal business logic, the same authorization layer can conceptually be placed in front of other REST API systems such as banking, healthcare, ERP, or HRMS applications without changing its core decision logic.

WORKFLOW / ARCHITECTURE DIAGRAM:

The diagram below illustrates the request lifecycle implemented by the ZTCA middleware, from the moment a user acts in the User Portal to the moment the Admin Security Dashboard reflects that action.

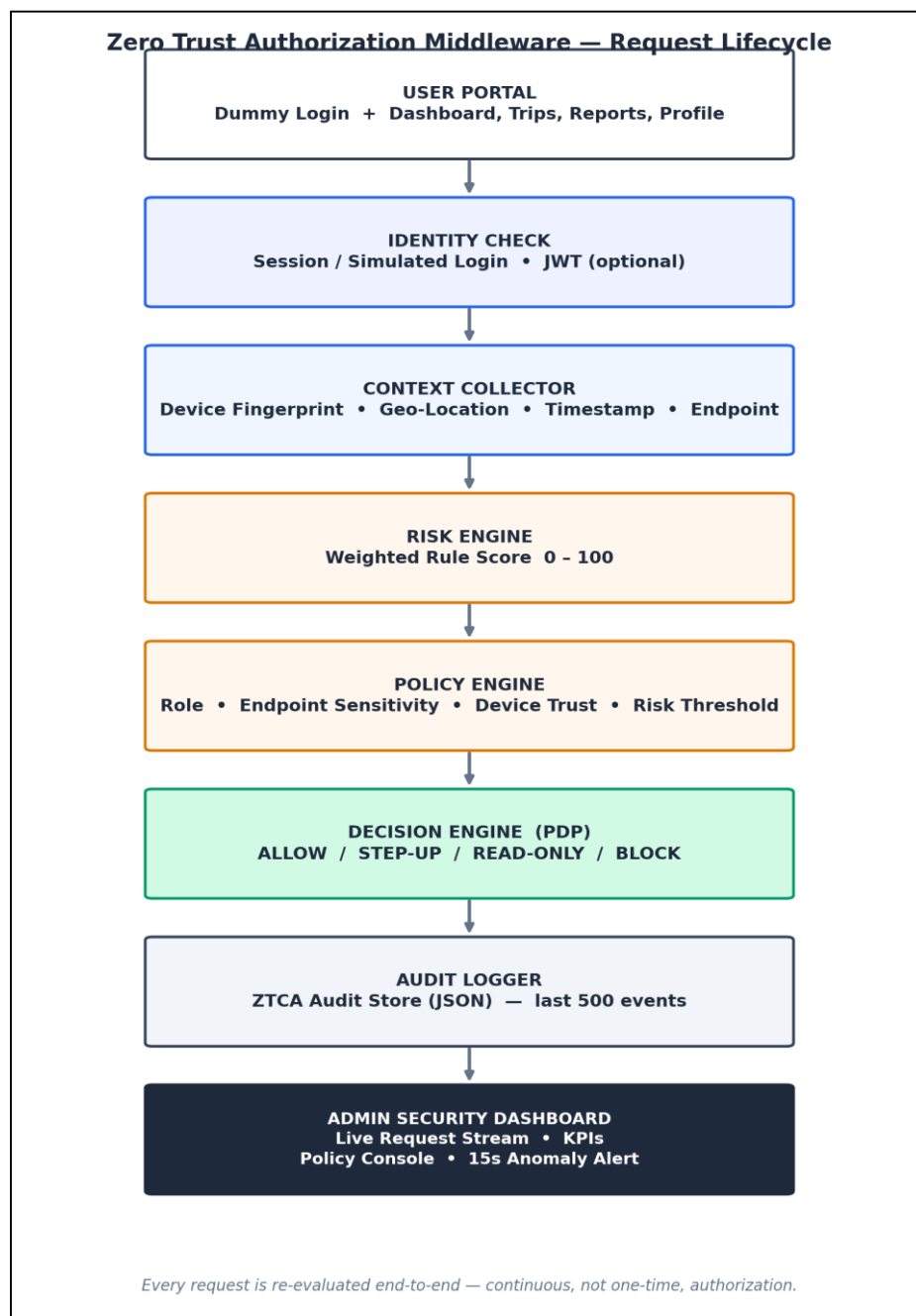


Fig 1: Zero Trust Authorization Middleware — Request Lifecycle

Explanation:

1. **User Interaction:** A user opens the TransitOps User Portal with a simulated (dummy) login, selecting a role such as Fleet Manager, Driver, Safety Officer, or Financial Analyst, and performs a normal action (viewing trips, dispatching a route, editing a driver record).
2. **Identity Check:** The request carries simulated identity headers (user id, name, role). A real JWT/OAuth layer can be dropped in later without touching any downstream module — identity and authorization are deliberately decoupled, as NIST SP 800-207 recommends.
3. **Context Collector:** The Express middleware captures device fingerprint, browser/OS, geo-location (or a manually entered simulation value), the exact timestamp, and the endpoint/method being called — for every request, not just at login.
4. **Risk Engine:** A plain-JS weighted function converts that context into a 0–100 risk score: unrecognized device +25, geographic anomaly +20, off-hours access +15, sensitive dispatch/CRUD action +20, role/privilege mismatch +25.
5. **Policy Engine:** Externally defined, admin-editable rules (role, endpoint sensitivity, device trust, risk threshold) are evaluated against the score — the same “policy separated from enforcement” pattern used by Open Policy Agent and AWS IAM.
6. **ZTCA Decision Engine:** Policy results and risk score are combined into one outcome — ALLOW, STEP-UP (MFA), READ-ONLY, or BLOCK — with a human-readable reason attached.
7. **Audit Logger:** Every decision, real or simulated, is written to a JSON-backed audit store (capped at the most recent 500 events) and made available to the Admin Security Dashboard.
8. **Admin Security Dashboard:** The dashboard polls the audit/metrics endpoints and also listens on a Broadcast Channel for instant pushes, so a Driver's action in one browser tab is reflected in the Admin's tab within 1–2 seconds, including a live 15-second anomaly alert toast.

Core components:

Component	Purpose
React application	Operational screens and simulation controls
ZTCAContextWidget	Context presets, dry-run risk forecast, factor preview
AdminPanel	Live security stream, KPI cards, analytics, policy and device administration
ztcaAuthorizationMiddleware	Request interception, context construction, audit logging, enforcement
ZTCA Decision Engine	Risk scoring and policy decision point
ZTCAAuditRepository	Persistent ordered audit events, maximum 500 records in demo mode
JSONStore	Simple file-backed repository abstraction

Never Trust, Always Verify - Least Privilege - Continuous, Context-Aware Authorization

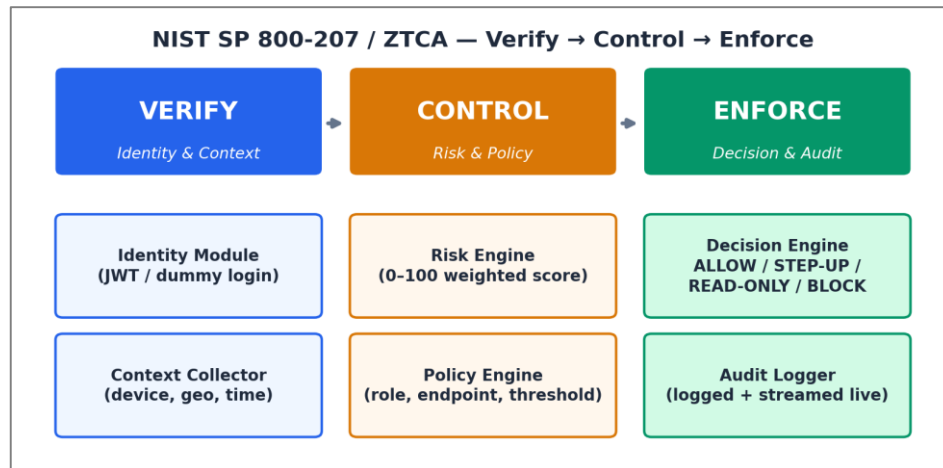


Fig 2: Module mapping to the NIST SP 800-207 / ZTCA Verify → Control → Enforce pillars

TECHNIQUES USED:

- **Frontend:** React 19 with Vite for the single-page application shell; Tailwind CSS for the enterprise interface; Framer Motion (`motion/react`) for panel and modal transitions; Recharts for KPI and analytics charts; and Lucide React for iconography.
- **Backend:** Node.js with Express as the API entry point, implemented in TypeScript. `tsx` is used for development execution and `esbuild` is used to bundle the production server into a standalone build.
- **ZTCA Authorization Layer:** Purpose-built TypeScript modules including `engine.ts` for risk scoring and authorization decisions, `middleware.ts` for request interception and enforcement, `store.ts` for audit and device persistence, and `types.ts` for shared security types. These modules are isolated under `backend/ztca/` to keep the security layer decoupled from fleet business logic.
- **API and Application Architecture:** Express routes handle business operations while the ZTCA authorization middleware evaluates protected requests before route handlers execute.
- **Persistence:** A lightweight Repository Pattern backed by JSON files under `backend/data/` for operational records, devices, locations, policies, and audit logs. The abstraction is designed to allow migration to SQLite or PostgreSQL without changing routes or views.
- **Real-Time Reflection:** Short-interval API polling combined with the same-origin `BroadcastChannel('ztca-updates')` mechanism for immediate KPI updates and cross-tab event reflection.
- **Security Frameworks and References:** NIST SP 800-207 Zero Trust Architecture, particularly the Policy Decision Point (PDP) and Policy Enforcement Point (PEP) model, together with the Zscaler ZTCA Verify → Control → Enforce framework. Open Policy Agent concepts were also used as a reference for separating policy from enforcement.
- **Development and Build Tools:** Vite, TypeScript, `tsx`, and `esbuild` for development, bundling, and production delivery.

METHODOLOGY

The project followed a structured, feasibility-first approach covering security requirements, architecture, implementation, validation, and deployment readiness.

1. Requirement Analysis, Feasibility & Zero Trust Scoping

- Identified fleet roles, sensitive operations, security risks, and Zero Trust requirements.
- Defined continuous request-level authorization, risk scoring, auditing, simulation, and real-time reflection as core requirements.
- Scoped ZTCA middleware as the primary contribution, with TransitOps as the demonstration platform.

2. System & Security Architecture Design

- Designed the flow from React and Express APIs through context collection, risk evaluation, policy enforcement, and auditing.
- Separated identity, risk, policy, decision, enforcement, and persistence components.
- Used the Repository Pattern and mapped the architecture to NIST Zero Trust concepts.

3. Incremental Implementation of the ZTCA Pipeline

- Built the TransitOps application, simulated identity, and request context collection.
- Implemented weighted risk scoring, configurable policies, and four authorization outcomes: ALLOW, STEP-UP, READ-ONLY, and BLOCK.
- Added audit logging for security and operational decisions.

4. Application, Administration & Security Observability

- Developed the TransitOps operational interfaces and ZTCA simulation controls.
- Built the Admin Security Panel with live requests, KPIs, risk inspection, policies, and device management.
- Added polling and BroadcastChannel-based updates for security event reflection.

5. Simulation, Testing & Security Validation

- Tested normal, elevated-risk, unprivileged, administrative, and critical-risk scenarios.
- Validated risk decisions, audit records, policy behavior, and forced 100/100 blocking.
- Verified simulation reflection, anomaly alerts, and administrative event tracking.

6. Hardening, Deployment Readiness & Documentation

- Resolved environment-specific issues and validated development and production builds.
- Documented implementation limitations and verified that security features remained functional after fixes.
- Defined future upgrades including database migration, real identity integration, WebSockets/SSE, MFA, and ML-assisted detection.

Development of the ZTCA Middleware:

Phase 1. Skeleton, Dummy Identity & Core Fleet App

Built TransitOps as the traffic-generating host application with modules for vehicles, drivers, trips, fuel, maintenance logs, expenses, notifications, and activity logs using the Repository Pattern with JSON-file persistence.

- Implemented a role-selector dummy login for Fleet Manager, Driver, Safety Officer, and Financial Analyst, attaching simulated identity information to outgoing requests through ZTCA headers.
- Established the initial React + Express application structure and operational workflows required to generate realistic protected API traffic.

The image displays three screenshots of the TransitOps application interface.

Left Screenshot: Secure Console Login

The login form includes fields for OPERATOR EMAIL (placeholder: name@transitops.com) and SECURE PASSWORD (masked with asterisks). A green "Enter Platform" button is present. Below the login form, a section titled "MULTI-USER EQUAL-PRIVILEGE EVALUATION PRESETS" lists several roles with their respective names:

- Super User (Admin):** ZTCA Security Administrator (admin@transitops.com)
- Fleet Manager #1:** Sarah Jenkins
- Fleet Manager #2:** David Miller
- Driver #1:** Alex Carter
- Driver #2:** Carlos Mendez
- Safety Officer #1:** Marcus Vance
- Safety Officer #2:** Rachel Chen
- Financial Analyst #1:** Elena Rostova
- Financial Analyst #2:** Robert Sterling

Right Screenshot: Platform Operator Registration

The registration form includes fields for FULL NAME (placeholder: e.g., Jane Doe), OPERATOR EMAIL (placeholder: jane@transitops.com), and SECURE PASSWORD (masked). A dropdown menu for SYSTEM ROLE is set to "Driver". Below these fields, a section titled "REQUIRED DRIVER INFORMATION" includes fields for Age (placeholder: e.g., 28), Gender (dropdown: Male), Driving Exp (Years) (placeholder: e.g., 5), and Mode of Work (dropdown: Simple Loads). A section for "LICENSE & ID DOCUMENT (IMAGE, PDF, DOC)" features a "Click to upload" button and a note "or drag & drop PDF, PNG, JPG, or DOC". A final field for "Place of Work - City (Optional)" has a placeholder "e.g., Chicago".

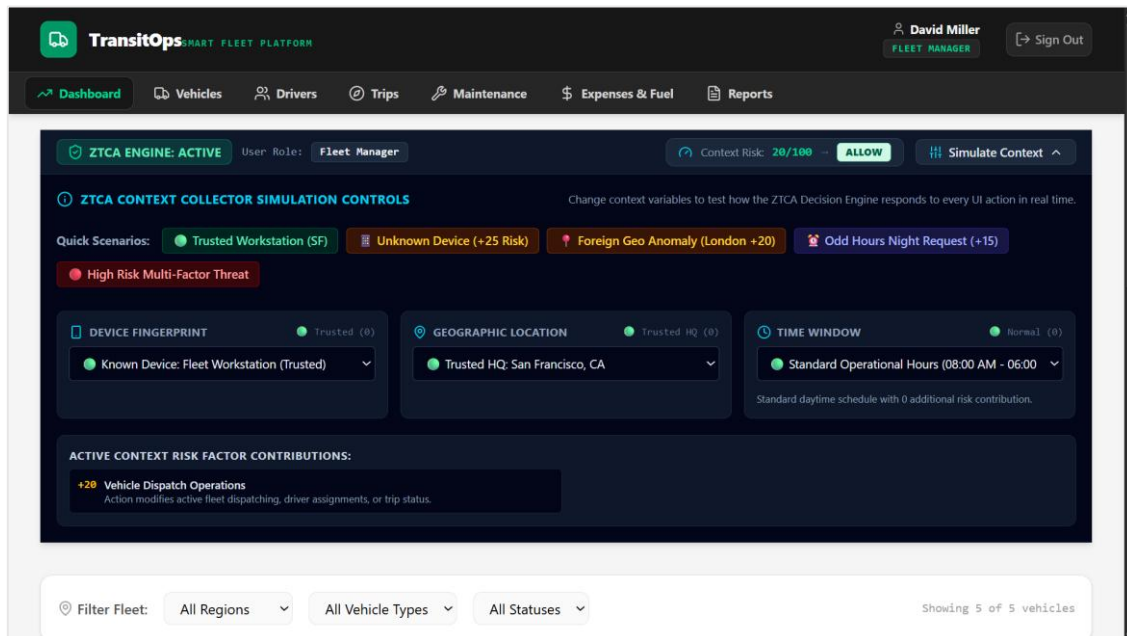
Bottom Screenshot: TransitOps Dashboard

The dashboard shows the user is logged in as David Miller (Fleet Manager). The top navigation bar includes links for Dashboard, Vehicles, Drivers, Trips, Maintenance, Expenses & Fuel, and Reports. The main content area displays various metrics and alerts:

- ZTCA ENGINE: ACTIVE** (User Role: Fleet Manager)
- Context Risk: 20/100** (ALLOW button)
- Simulate Context** (dropdown menu)
- Filter Fleet:** All Regions, All Vehicle Types, All Statuses (Showing 5 of 5 vehicles)
- ACTIVE VEHICLES:** 1 (Currently on delivery)
- AVAILABLE FLEET:** 2 (Ready for dispatch)
- IN SHOP / MAINTENANCE:** 1 (Scheduled repairs)
- FLEET UTILIZATION:** 25% (On road vs active total)
- DRIVERS ON DUTY:** 1
- ACTIVE DISPATCHED TRIPS:** 1
- PENDING DRAFT TRIPS:** 1
- Compliance & Safety Alerts:** 4 Pending (License Expiry: Driver Robert Chen's license (DL-8271034) expired on 2026-06-30)
- Live Fleet Activity Feed:** User David Miller logged in. (1:43:15 PM), User Sarah Jenkins logged in. (1:43:03 PM)

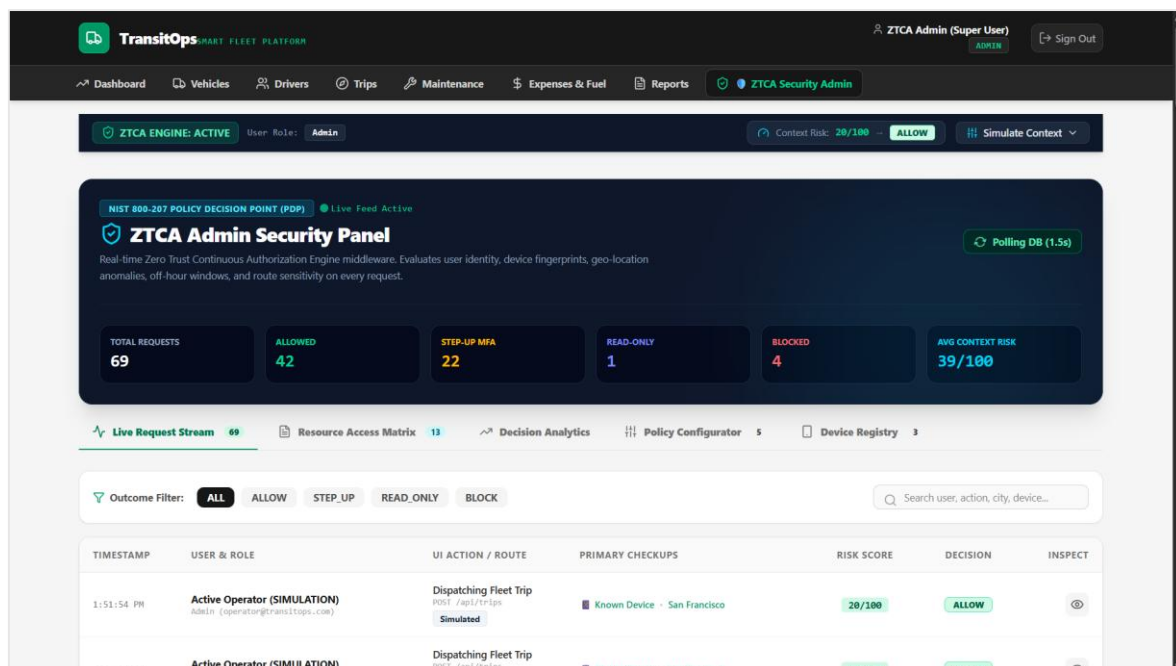
Phase 2. Context Collector, Risk Engine & Policy Engine

- Implemented request-level context collection for device fingerprint, browser/OS, geographic location, timestamp, endpoint, method, and user role.
- Developed an explainable 0–100 weighted risk engine and configurable Policy Engine using role, endpoint sensitivity, device trust, and risk thresholds.
- Added simulation controls for custom devices, locations, and off-hours conditions, together with device trust management).



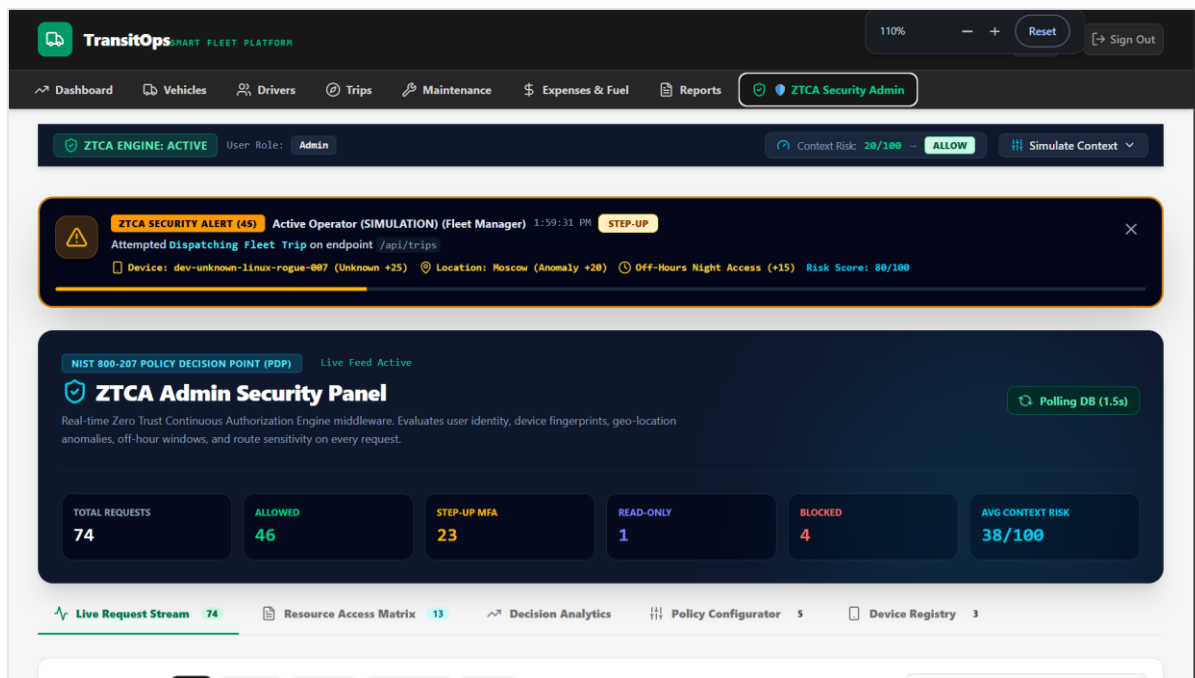
Phase 3. Decision Engine, Audit Logger & Admin Security Panel

- Connected identity, context, risk, and policy evaluation through the ZTCA authorization middleware to produce *ALLOW*, *STEP-UP*, *READ-ONLY*, or *BLOCK* decisions.
- Added JSON-backed audit persistence for authorization events and administrative actions, with a demonstration limit of 500 records.
- Built the Admin Security Panel with live request stream, KPI cards, risk inspection, policy configuration, and device registry with trust/revoke controls.



Phase 4. Simulation Hardening, Multi-Tab Reflection & Live Alerts

- Added auditable tracking for administrative actions and forced 100/100 simulations to produce a hard BLOCK tagged SIMULATION_FORCED_BLOCK.
- Connected simulation metrics to the Admin Panel through API updates and Broadcast Channel ('ztca-updates'), allowing critical KPI changes to appear immediately across open tabs.
- Added a dismissible 15-second anomaly alert for unknown devices, anomalous locations, off-hours activity, and high-risk requests.



Responses Achieved:

The completed TransitOps system successfully demonstrated continuous, context-aware authorization across realistic fleet operations. The resulting interfaces provided operational workflows alongside security visibility through the ZTCA Admin Security Panel, including live request monitoring, risk inspection, policy management, device trust controls, and real-time security event reflection.

ZTCA Principles Achieved:

ZTCA Principle	Implementation Demonstrated	Result Achieved
1. Verify who is connecting	User authentication, role identification, and administrative identity checks	Requests were associated with authenticated users and their assigned roles.
2. Verify attributes	Evaluation of user role, device identity, location, time, request type, and endpoint sensitivity	Authorization decisions considered multiple attributes rather than identity alone.
3. Verify where the connection is going	Route, resource, and operation-level evaluation	Access was evaluated according to the requested API endpoint and action.
4. Assess risk using context	Contextual risk scoring based on device, location, time, privilege, and request sensitivity	Requests were classified according to their combined contextual risk.

5. Prevent compromise	Unknown-device detection, anomaly detection, step-up verification, and high-risk blocking	Suspicious requests were challenged, restricted, or denied before causing harm.
6. Prevent data loss	Read-only restrictions, sensitive-operation controls, and blocking of unauthorized modifications	Risky write operations were prevented while lower-risk access could remain available.
7. Enforce policy	Policy-based decisions producing ALLOW, STEP-UP, READ-ONLY, or BLOCK outcomes	Security rules were applied consistently and recorded for audit and review.

Representative Security Responses:

- **ALLOW:** Normal requests from trusted contexts proceeded and were recorded in the audit stream.
- **STEP-UP:** Elevated-risk sensitive actions required additional verification before proceeding.
- **READ-ONLY:** Moderate-risk write operations were restricted while read access remained available.
- **BLOCK:** Unprivileged, critical-risk, or policy-violating requests were denied with an auditable reason.
- **Forced 100/100 Simulation:** Critical simulated risk was automatically converted to BLOCK, tagged SIMULATION_FORCED_BLOCK, and reflected immediately in the security metrics.

All decisions, including simulated and administrative events, were persisted and made visible through the Admin Security Panel. This demonstrated that the seven ZTCA principles were not only defined conceptually but implemented through verification, contextual risk assessment, compromise and data-loss prevention, and consistent policy enforcement.

Representative Test Scenarios:

Case	Test context	Expected result in Admin Panel
Allow	Known user, trusted device, verified location, normal hours, low-risk read	ALLOW row and Allowed count increment
Step-Up MFA	Fleet Manager dispatch from unknown device or off-hours	STEP_UP row, challenge visible, Step-Up count increment
Read-Only	Moderate-risk write request above read-only threshold	READ_ONLY row, reason and factors visible
Unprivileged block	Driver attempts admin or restricted operation	BLOCK row, actor and privilege mismatch visible
Suspension action	Manager changes driver status to Suspended	Administrative audit row identifies actor and blocked status action
100/100 simulation	Unknown device + foreign location + odd-hours + sensitive endpoint + privilege mismatch	Forced BLOCK, Simulated marker, Blocked count increment immediately
Policy change	Admin enables, disables, creates, or updates a rule	Audit row identifies policy action and acting administrator

Every response above is written to the ZTCA audit store and appears in the Admin Security Panel's live stream in real time, whether the action originated from genuine user traffic or from the simulation widget (clearly tagged as such).

Reflection rules:

- Simulation rows show a Simulated marker.
- Blocked simulation decisions show an explicit SIMULATION FORCED BLOCK reason.
- Blocked KPI is calculated from persisted audit decisions, including simulations.
- Administrative driver/device/user/policy actions are logged with actor information when available.

The Admin Panel refreshes through DB polling and receives an immediate simulation metrics broadcast

Demo persistence behaviour:

- Operational data is stored in backend/data JSON files.
- ZTCA policies, devices, locations, and audit logs are persisted separately.
- Audit entries are sorted newest first and retained up to 500 entries.
- The JSON repository is appropriate for local demonstration and testing; migrate to a transactional database for production.

Use Cases:

- **Fleet Operations:** Secure vehicle, driver, trip, and expense APIs.
- **Enterprise Systems:** Protect sensitive ERP, HRMS, and internal APIs.
- **Financial Services:** Apply continuous authorization to high-value transactions.
- **Healthcare Systems:** Evaluate access to sensitive patient and operational data.

Limitations:

- Identity and MFA are simulated rather than production-integrated.
- JSON files are used instead of a production database.
- Risk scoring is rule-based rather than ML-driven.
- Geolocation and device fingerprinting depend on browser/device capabilities.
- Polling is used for live updates instead of WebSockets or SSE.

Future Enhancements:

- Integrate **OAuth/JWT and real MFA**.
- Migrate persistence to **PostgreSQL/SQLite**.
- Replace polling with **WebSockets/SSE**.
- Add **ML-based anomaly and risk detection**.
- Extend toward production-scale Zero Trust deployments.

CONCLUSION:

The internship provided valuable practical experience in **Zero Trust security, context-aware authorization, REST API protection, and full-stack application development**. Through the development of ZTCA Middleware and its demonstration through TransitOps, I gained hands-on experience in designing request-level authorization, risk-based decisions, policy enforcement, auditing, and security monitoring.

The project also provided practical exposure to translating **Zero Trust principles into a working application**, from architecture and implementation to simulation, testing, and deployment readiness. Overall, the internship strengthened my understanding of secure software design and prepared me to work on **scalable, context-aware, and security-focused applications**.

Zero Trust & Security References

1. NIST SP 800-207 - Zero Trust Architecture | [NIST SP 800-207](#)

Defines Zero Trust as an architecture that removes implicit trust and focuses access decisions on users, assets, resources, and context.

2. BeyondCorp: A New Approach to Enterprise Security - Google | [Google Research - BeyondCorp](#)

Demonstrates Google's shift from perimeter-based security toward access based on users, devices, and application context.

3. NIST SP 800-207A - Zero Trust for Cloud-Native Applications | [NIST SP 800-207A](#)

Extends Zero Trust principles to application-level access control using identities and granular policies.

4. BeyondCorp: The Access Proxy – Google | [Google Research - BeyondCorp Access Proxy](#)

Describes an access-proxy architecture for enforcing security policies across internal applications.

Architecture & Implementation References

5. NIST Zero Trust Architecture - Core Components | [NIST ZTA Project Documentation](#)

Describes the Policy Engine, Policy Administrator, and Policy Enforcement Point used to implement Zero Trust access decisions.

6. NIST Zero Trust Architecture Project

Provides practical architecture and implementation guidance for applying Zero Trust principles. NIST Zero Trust Architecture Project.